HARARE INSTITUTE OF TECHNOLOGY

OBJECT-ORIENTED PROGRAMMING IN JAVA

PRACTICAL LABORATORY REPORT

Student Name: Precious Millicent Zvidi
Registration Number: H250137J
Department: Software Engineering
Programme: BTech Honours in Software Engineering
Programming Language: Java
Development Environment: IntelliJ IDEA
Academic Year: 2026

---

1. INTRODUCTION

Object-Oriented Programming (OOP) is a programming approach that organiZes software around classes and objects. It helps developers create programs that are structured, reusable, maintainable and easier to understand. Java is an object-oriented programming language that supports important concepts such as encapsulation, inheritance, polymorphism and abstraction.

This report presents the practical activities covered in the Java OOP laboratory series, from Lab 1 to Lab 10, together with the Lab 11 mini-project. The activities progress from basic class creation to more advanced concepts such as exception handling, collections, file input and output, lambda expressions, streams and design patterns.

The practical exercises were intended to strengthen programming skills through writing, compiling, running and debugging Java programs using IntelliJ IDEA. The final mini-project, a Library Management System, combines several of the concepts studied throughout the laboratory series.

2. GENERAL OBJECTIVES

The main objectives of the laboratory series were:

1. To understand the fundamental principles of object-oriented programming in Java.
2. To create classes and objects and use their fields and methods.
3. To apply constructors, encapsulation, inheritance, polymorphism and abstraction.
4. To use interfaces and access modifiers appropriately.
5. To handle exceptions and develop more reliable programs.
6. To store and manage data using generics and Java collections.

7. To work with nested classes, enums, records and object methods.
8. To read and write files and process data using lambdas and streams.
9. To understand and apply common software design patterns.
10. To integrate Java programming concepts in a practical mini-project.

3. DEVELOPMENT ENVIRONMENT AND METHODOLOGY

3.1 Development Environment

The following tools and technologies were used for the practical activities:

- **Programming language:** Java
- **Integrated Development Environment:** IntelliJ IDEA
- **Java Development Kit:** The JDK installed for the practical exercises
- **Project organization:** Separate source folders for individual laboratory exercises
- **Version control and storage:** GitHub for storing and sharing project source files

3.2 Methodology

The practical work followed a step-by-step programming approach. Each exercise was studied to identify its objectives and required programming concepts. Java classes and methods were then created in IntelliJ IDEA. The programs were compiled and executed to observe their behaviour, and errors were corrected where necessary.

The displayed output was used to check whether a program behaved as expected. Source files were organised into the relevant laboratory folders to make the work easier to manage and review.

4. LAB 1: CLASSES AND OBJECTS

4.1 Objectives

The objectives of Lab 1 were to:

- Define classes containing fields and methods.
- Create objects using the `new` keyword.
- Access object members using the dot operator.
- Understand object references and the `null` value.
- Model relationships between objects.

4.2 Introduction

A class is a blueprint that defines the data and behaviour of a particular type of object. An object is an instance of a class. Fields represent the information stored by an object, while methods define the actions that it can perform.

For example, a `Student` class can contain a student's name, registration number and GPA. Each student object can have different values while using the same class structure.

4.3 Practical Work Performed

The practical exercises introduced the creation and use of Java classes. A `Student` class was used to demonstrate fields, methods and object creation. Objects were created and their details displayed using methods.

Further exercises involved creating a `Car` class with attributes such as brand, year and mileage. A method was used to display the details of each car, while another method could determine whether a car was more than 25 years old.

The `Rectangle` exercise introduced calculating area and perimeter. An array of rectangle objects could be used to compare their areas and identify the rectangle with the largest area. The relationship between `Person` and `Address` also demonstrated how one object can contain a reference to another object.

4.4 Results and Observations

The exercises demonstrated that several objects can be created from one class. Each object can store its own values and use the methods defined in the class.

It was also established that assigning one object reference to another does not automatically create a separate object. Both references may point to the same object. A `null` reference does not refer to an object, so attempting to access its members can cause a `NullPointerException`.

On Address.java the code is as below

```
package lab01;

public class Address {

    // Fields representing address details
    String street;
    String city;
    String country;

    // Constructor to initialise the address
    public Address(String street, String city, String country) {
        this.street = street;
```

```
        this.city = city;
        this.country = country;
    }
}
```

On Car.java the code is as below

```
package lab01;

import java.time.Year;

public class Car {

    // Fields describing a car
    String brand;
    int year;
    double mileage;

    // Exercise 1: Display car details
    void display() {
        System.out.println("Brand: " + brand);
        System.out.println("Year: " + year);
        System.out.println("Mileage: " + mileage + " km");
        System.out.println("--------------------");
    }

    // Exercise 2: Check whether a car is antique
    boolean isAntique() {
        int currentYear = Year.now().getValue();
        return (currentYear - year) > 25;
    }

    public static void main(String[] args) {

        // Create the first car
        Car car1 = new Car();
        car1.brand = "Toyota";
        car1.year = 1995;
        car1.mileage = 250000;

        // Create the second car
        Car car2 = new Car();
        car2.brand = "Honda";
        car2.year = 2015;
        car2.mileage = 120000;

        // Create the third car
        Car car3 = new Car();
        car3.brand = "Mercedes-Benz";
        car3.year = 1988;
        car3.mileage = 310000;
```

```
        // Display the three cars
        car1.display();
        car2.display();
        car3.display();

        // Check whether each car is antique
        System.out.println("Toyota antique? " + car1.isAntique());
        System.out.println("Honda antique? " + car2.isAntique());
        System.out.println("Mercedes-Benz antique? " + car3.isAntique());
    }
}
```

The output

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=58179" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab01.Car

Brand: Toyota

Year: 1995

Mileage: 250000.0 km

Brand: Honda

Year: 2015

Mileage: 120000.0 km

Brand: Mercedes-Benz

Year: 1988

Mileage: 310000.0 km

Toyota antique? true

Honda antique? false

Mercedes-Benz antique? true

Process finished with exit code 0

On the Person.java the code is as below

```
package lab01;

public class Person {

    // Person details
    String name;
    int age;

    // A Person HAS-A Address
    Address address;

    // Constructor
    public Person(String name, int age, Address address) {
        this.name = name;
        this.age = age;
        this.address = address;
    }

    public static void main(String[] args) {

        // Create an Address object
        Address homeAddress = new Address(
                "Samora Machel Avenue",
                "Harare",
                "Zimbabwe"
        );

        // Create a Person with an Address
        Person person1 = new Person(
                "Precious",
                21,
                homeAddress
        );

        // Chained dot expression
        System.out.println("Name: " + person1.name);
        System.out.println("City: " + person1.address.city);

        // Create another person without an address
        Person person2 = new Person(
                "Tariro",
                22,
```

```
                null
        );

        // Check for null before accessing the address
        if (person2.address == null) {
            System.out.println("Tariro has no address assigned.");
        }

        // Demonstrate what happens if null is accessed
        try {
            System.out.println(person2.address.city);
        } catch (NullPointerException e) {
            System.out.println("Error: Cannot access city because address is
null.");
        }
    }
}
```

Output

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=58182" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab01.Person

Name: Precious

City: Harare

Tariro has no address assigned.

Error: Cannot access city because address is null.

Process finished with exit code 0

On Rectangle.java the code is as below

```
package lab01;

public class Rectangle {

    // Fields
    double width;
    double height;
```

```
    // Calculate area
    double area() {
        return width * height;
    }

    // Calculate perimeter
    double perimeter() {
        return 2 * (width + height);
    }

    public static void main(String[] args) {

        // Create an array of five rectangles
        Rectangle[] rectangles = new Rectangle[5];

        // Create and initialise each rectangle
        for (int i = 0; i < rectangles.length; i++) {
            rectangles[i] = new Rectangle();
            rectangles[i].width = (i + 1) * 2;
            rectangles[i].height = (i + 1) * 3;
        }

        // Assume the first rectangle has the largest area
        Rectangle largest = rectangles[0];

        // Display rectangles and find the largest
        for (Rectangle r : rectangles) {
            System.out.println("Width: " + r.width);
            System.out.println("Height: " + r.height);
            System.out.println("Area: " + r.area());
            System.out.println("Perimeter: " + r.perimeter());
            System.out.println("---------------------");

            if (r.area() > largest.area()) {
                largest = r;
            }
        }

        // Display rectangle with largest area
        System.out.println("Rectangle with largest area:");
        System.out.println("Width: " + largest.width);
        System.out.println("Height: " + largest.height);
        System.out.println("Area: " + largest.area());
    }
}
```

Output

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56444" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab01.Rectangle

Width: 2.0

Height: 3.0

Area: 6.0

Perimeter: 10.0

Width: 4.0

Height: 6.0

Area: 24.0

Perimeter: 20.0

On RectangleReference.java the code is as below

```
package lab01;

public class RectangleReferenceDemo {

    public static void main(String[] args) {

        // Create the first rectangle
        Rectangle r1 = new Rectangle();
        r1.width = 10;
        r1.height = 5;

        // Assign r1's reference to r2
        Rectangle r2 = r1;

        // Change width using r2
        r2.width = 20;

        // Display the values
        System.out.println("Width of r1: " + r1.width);
        System.out.println("Width of r2: " + r2.width);

        // Check whether both refer to the same object
        System.out.println("Same object? " + (r1 == r2));
    }
}
```

Output

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=61929" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab01.RectangleReferenceDemo

Width of r1: 20.0

Width of r2: 20.0

Same object? true

Process finished with exit code 0

4.5 Conclusion

Lab 1 established the foundation of Java object-oriented programming. It demonstrated how classes and objects represent real-world entities and how fields, methods and references work together in a Java program.

5. LAB 2: CONSTRUCTORS, THE THIS KEYWORD AND STATIC MEMBERS

5.1 Objectives

The objectives of Lab 2 were to:

- Initialise objects using constructors.
- Apply constructor overloading and constructor chaining.
- Use the `this` keyword.
- Distinguish between instance and static members.
- Understand static and instance initialisation blocks.

5.2 Introduction

A constructor is a special part of a class that is executed when an object is created. It is used to initialise the object's fields. Constructor overloading allows a class to have more than one constructor with different parameter lists.

The `this` keyword refers to the current object. Static members belong to the class itself, while instance members belong to individual objects.

5.3 Practical Work Performed

The practical exercises used a `Book` class to demonstrate constructors that accepted different combinations of information. Constructor chaining was used to reuse initialisation logic rather than repeating the same statements.

The `this` keyword was used to distinguish instance fields from constructor parameters with the same names. A static counter demonstrated how a class can keep track of the number of objects created.

Initialisation blocks were also examined to understand the order in which static fields, static blocks, instance fields, instance blocks and constructors execute.

5.4 Results and Observations

The exercises showed that constructors initialise objects when they are created. Constructor overloading provides flexibility when different amounts of information are available.

Static members are shared by objects of the same class, whereas each object has its own instance fields. Static initialisation occurs when the class is initialised, while instance initialisation occurs for each new object.

Circle.java

```
package lab02;

public class Circle {

    // Instance variable: each circle has its own radius
    private double radius;

    // Static variable: shared by all Circle objects
    private static int count = 0;

    // Exercise 1: Constructor accepting a radius
    public Circle(double radius) {
        this.radius = radius;
        count++;
    }

    // Exercise 1: No-argument constructor
    public Circle() {
        this(1.0); // Calls the constructor above
    }

    // Exercise 2: Static getter for circle count
    public static int getCount() {
        return count;
    }

    // Calculate area
    public double area() {
        return Math.PI * radius * radius;
```

```
    }

    // Display circle information
    public void display() {
        System.out.println("Radius: " + radius);
        System.out.printf("Area: %.2f%n", area());
        System.out.println("--------------------");
    }

    public static void main(String[] args) {

        // Create circle using parameterised constructor
        Circle circle1 = new Circle(5.0);

        // Create circle using no-argument constructor
        Circle circle2 = new Circle();

        // Create another circle
        Circle circle3 = new Circle(3.0);

        // Display details
        circle1.display();
        circle2.display();
        circle3.display();

        // Display total number of circles
        System.out.println(
                "Total circles created: " + Circle.getCount()
        );
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56292" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab02.Circle

Radius: 5.0

Area: 78.54

Radius: 1.0

Area: 3.14

Radius: 3.0

Area: 28.27

Total circles created: 3

Process finished with exit code 0

Employee.java

```
package lab02;

public class Employee {

    // Static counter shared by all employees
    private static int employeeCount = 0;

    // Employee details
    private final int employeeId;
    private String name;
    private String department;
    private double salary;

    // Constructor 1: No arguments
    public Employee() {
        this("Unknown", "Not Assigned", 0.0);
    }

    // Constructor 2: Name only
    public Employee(String name) {
        this(name, "Not Assigned", 0.0);
    }

    // Constructor 3: Full employee details
    public Employee(String name, String department, double salary) {
        employeeCount++;
        this.employeeId = employeeCount;
        this.name = name;
        this.department = department;
        this.salary = salary;
    }

    // Copy constructor
    // Creates a new employee with copied details and a new ID
    public Employee(Employee other) {
        this(other.name, other.department, other.salary);
    }

    // Display employee information
    public void display() {
        System.out.println("Employee ID: " + employeeId);
        System.out.println("Name: " + name);
```

```
        System.out.println("Department: " + department);
        System.out.println("Salary: " + salary);
        System.out.println("--------------------");
    }

    public static void main(String[] args) {

        // Using the no-argument constructor
        Employee emp1 = new Employee();

        // Using the name-only constructor
        Employee emp2 = new Employee("Precious");

        // Using the full constructor
        Employee emp3 = new Employee(
                "Tendai", "Software Development", 1500.00
        );

        // Using the copy constructor
        Employee emp4 = new Employee(emp3);

        // Display all employees
        emp1.display();
        emp2.display();
        emp3.display();
        emp4.display();

        System.out.println(
                "Total employees created: " + employeeCount
        );
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56295" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab02.Employee

Employee ID: 1

Name: Unknown

Department: Not Assigned

Salary: 0.0

Employee ID: 2

Name: Precious

Department: Not Assigned

Salary: 0.0

Employee ID: 3

Name: Tendai

Department: Software Development

Salary: 1500.0

Employee ID: 4

Name: Tendai

Department: Software Development

Salary: 1500.0

Total employees created: 4

Process finished with exit code 0

InitializationDemo.java

```
package lab02;

public class InitializationDemo {

    // First static initialiser block
    static {
        System.out.println("Static Block 1");
    }

    // Second static initialiser block
    static {
        System.out.println("Static Block 2");
```

```
    }

    // First instance initialiser block
    {
        System.out.println("Instance Block 1");
    }

    // Second instance initialiser block
    {
        System.out.println("Instance Block 2");
    }

    // Constructor
    public InitializationDemo() {
        System.out.println("Constructor Executed");
    }

    public static void main(String[] args) {

        System.out.println("Creating First Object:");
        InitializationDemo obj1 = new InitializationDemo();

        System.out.println("Creating Second Object:");
        InitializationDemo obj2 = new InitializationDemo();
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56297" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab02.InitializationDemo

Static Block 1

Static Block 2

Creating First Object:

Instance Block 1

Instance Block 2

Constructor Executed

Creating Second Object:

Instance Block 1

Instance Block 2

Constructor Executed

Process finished with exit code 0

Temperature.java

```
package lab02;

public class Temperature {
    // Store temperature internally in Celsius
    private final double celsius;

    // Private constructor prevents direct object creation
    private Temperature(double celsius) {
        this.celsius = celsius;
    }

    // Static factory method for Celsius
    public static Temperature fromCelsius(double celsius) {
        return new Temperature(celsius);
    }

    // Static factory method for Fahrenheit
    public static Temperature fromFahrenheit(double fahrenheit) {
        double celsius = (fahrenheit - 32) * 5.0 / 9.0;
        return new Temperature(celsius);
    }

    // Return temperature in Celsius
    public double getCelsius() {
        return celsius;
    }

    // Convert Celsius to Fahrenheit
    public double getFahrenheit() {
        return (celsius * 9.0 / 5.0) + 32;
    }

    // Display temperature
    public void display() {
        System.out.printf("Celsius: %.2f°C%n", getCelsius());
        System.out.printf("Fahrenheit: %.2f°F%n", getFahrenheit());
        System.out.println("--------------------");
    }

    public static void main(String[] args) {

        // Create temperature from Celsius
        Temperature temp1 = Temperature.fromCelsius(25);

        // Create temperature from Fahrenheit
        Temperature temp2 = Temperature.fromFahrenheit(98.6);
```

```
        System.out.println("Temperature 1:");
        temp1.display();

        System.out.println("Temperature 2:");
        temp2.display();
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56299" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab02.Temperature

Temperature 1:

Celsius: 25.00°C

Fahrenheit: 77.00°F

Temperature 2:

Celsius: 37.00°C

Fahrenheit: 98.60°F

Process finished with exit code 0

5.5 Conclusion

Lab 2 demonstrated how constructors support object initialisation and how the `this` keyword and static members help organise Java classes.

6. LAB 3: ENCAPSULATION AND ACCESS MODIFIERS

6.1 Objectives

The objectives of Lab 3 were to:

- Protect internal class data.
- Apply Java access modifiers correctly.
- Use getters and setters to control access to fields.
- Validate data before accepting changes.
- Prevent objects from entering invalid states.

6.2 Introduction

Encapsulation is the principle of combining data and the methods that operate on that data within a class while controlling access to the internal details. It helps prevent unauthorised or invalid changes to an object's state.

Java provides access modifiers such as `private`, `protected` and `public`. These modifiers determine where fields, methods and other class members can be accessed.

6.3 Practical Work Performed

A `BankAccount` example was used to demonstrate encapsulation. Fields such as the account number, owner's name and balance were protected from direct external access.

Getter methods provided controlled access to information, while methods such as `deposit()` and `withdraw()` managed changes to the account balance. Validation rules were used to reject invalid amounts and prevent withdrawals exceeding the available balance.

The exercises also demonstrated how exposing a mutable internal collection can allow other code to change an object's data unexpectedly.

6.4 Results and Observations

The practical work demonstrated that private fields help protect an object's internal state. Validation methods can reject invalid data before it affects the object.

For example, a deposit method should reject zero or negative amounts. A withdrawal method should also check whether sufficient funds are available before reducing the balance.

BankAccount.java

```java
package lab03;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class TransactionAccount {
    // Private fields
    private String accountHolder;
    private double balance;

    // Private transaction history
    private final List<String> transactions = new ArrayList<>();

    // Constructor
    public TransactionAccount(String accountHolder, double balance) {
        this.accountHolder = accountHolder;

        if (balance >= 0) {
            this.balance = balance;
        } else {
            this.balance = 0;
        }

        transactions.add("Opening balance: $" + this.balance);
    }

    // Deposit money
    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            transactions.add("Deposited: $" + amount);
            System.out.println("Deposit successful!");
        } else {
            System.out.println("Invalid deposit amount!");
        }
    }

    // Withdraw money
    public void withdraw(double amount) {
        if (amount <= 0) {
            System.out.println("Invalid withdrawal amount!");
        } else if (amount > balance) {
            System.out.println("Insufficient funds!");
        } else {
            balance -= amount;
            transactions.add("Withdrawn: $" + amount);
            System.out.println("Withdrawal successful!");
        }
    }

    // Getter for balance
```

```
    public double getBalance() {
        return balance;
    }

    // Return a read-only view of transaction history
    public List<String> getTransactions() {
        return Collections.unmodifiableList(transactions);
    }

    // Display account information
    public void display() {
        System.out.println("Account Holder: " + accountHolder);
        System.out.println("Balance: $" + balance);
    }

    // Display transaction history
    public void displayTransactions() {
        System.out.println("\nTransaction History:");

        for (String transaction : transactions) {
            System.out.println(transaction);
        }
    }

    public static void main(String[] args) {

        TransactionAccount account =
                new TransactionAccount("Precious", 500);

        account.deposit(200);
        account.withdraw(100);
        account.deposit(300);
        account.withdraw(50);

        account.display();
        account.displayTransactions();
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56302" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab03.BankAccount

Initial Account Details:

Account Holder: Precious

Current Balance: $500.0

Deposited: $200.0

Withdrawn: $100.0

Invalid deposit amount!

Insufficient funds!

Final Account Details:

Account Holder: Precious

Current Balance: $600.0

Process finished with exit code 0

ImmutableStudent.java

```
package lab03;

// final prevents other classes from inheriting this class
public final class ImmutableStudent {
    // Private final fields
    private final String name;
    private final String registrationNumber;
    private final String department;

    // Constructor initializes all fields
    public ImmutableStudent(String name,
                            String registrationNumber,
                            String department) {

        this.name = name;
        this.registrationNumber = registrationNumber;
        this.department = department;
    }

    // Getter methods only
    public String getName() {
        return name;
    }

    public String getRegistrationNumber() {
        return registrationNumber;
    }

    public String getDepartment() {
        return department;
```

```
    }

    // Display student information
    public void display() {
        System.out.println("Name: " + name);
        System.out.println("Registration Number: "
                + registrationNumber);
        System.out.println("Department: " + department);
    }

    public static void main(String[] args) {

        // Create an immutable student object
        ImmutableStudent student = new ImmutableStudent(
                "Precious Millicent Zvidi",
                "H250137J",
                "Software Engineering"
        );

        student.display();

        // The following would cause a compilation error:
        // student.name = "Another Name";
        // student.setName("Another Name");
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56304" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab03.ImmutableStudent

Name: Precious Millicent Zvidi

Registration Number: H250137J

Department: Software Engineering

Process finished with exit code 0

Student.java

```
package lab03;

public class Student {
    // Private fields: cannot be accessed directly outside the class
    private String name;
    private int age;
```

```
    private double grade;

    // Constructor
    public Student(String name, int age, double grade) {
        this.name = name;
        setAge(age);
        setGrade(grade);
    }

    // Getter for name
    public String getName() {
        return name;
    }

    // Setter for name
    public void setName(String name) {
        this.name = name;
    }

    // Getter for age
    public int getAge() {
        return age;
    }

    // Setter with validation
    public void setAge(int age) {
        if (age > 0) {
            this.age = age;
        } else {
            System.out.println("Invalid age!");
        }
    }

    // Getter for grade
    public double getGrade() {
        return grade;
    }

    // Setter with validation
    public void setGrade(double grade) {
        if (grade >= 0 && grade <= 100) {
            this.grade = grade;
        } else {
            System.out.println("Invalid grade!");
        }
    }

    // Display student information
    public void display() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Grade: " + grade);
    }

    public static void main(String[] args) {

        Student student1 = new Student(
```

```
                "Precious", 20, 85.5
        );

        System.out.println("Original Student Details:");
        student1.display();

        // Update values using setters
        student1.setAge(21);
        student1.setGrade(90);

        System.out.println("\nUpdated Student Details:");
        student1.display();

        // Test validation
        student1.setAge(-5);
        student1.setGrade(120);
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56306" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab03.Student

Original Student Details:

Name: Precious

Age: 20

Grade: 85.5

Updated Student Details:

Name: Precious

Age: 21

Grade: 90.0

Invalid age!

Invalid grade!

Process finished with exit code 0

TemperatureConverter.java

```java
package lab03;

public class TemperatureConverter {

    // Private field to store temperature in Celsius
    private double celsius;

    // Constructor
    public TemperatureConverter(double celsius) {
        setCelsius(celsius);
    }

    // Getter for Celsius
    public double getCelsius() {
        return celsius;
    }

    // Setter with validation
    public void setCelsius(double celsius) {
        if (Double.isFinite(celsius) && celsius >= -273.15) {
            this.celsius = celsius;
        } else {
            System.out.println("Invalid temperature!");
        }
    }

    // Convert Celsius to Fahrenheit
    public double getFahrenheit() {
        return (celsius * 9.0 / 5.0) + 32;
    }

    // Set temperature using Fahrenheit
    public void setFahrenheit(double fahrenheit) {
        if (!Double.isFinite(fahrenheit)) {
            System.out.println("Invalid temperature!");
            return;
        }

        double converted = (fahrenheit - 32) * 5.0 / 9.0;
        setCelsius(converted);
    }

    // Display temperature
    public void display() {
        System.out.printf("Celsius: %.2f°C%n", getCelsius());
        System.out.printf("Fahrenheit: %.2f°F%n", getFahrenheit());
    }

    public static void main(String[] args) {
```

```
        TemperatureConverter temp =
                new TemperatureConverter(25);

        System.out.println("Original Temperature:");
        temp.display();

        // Update using Fahrenheit
        temp.setFahrenheit(98.6);

        System.out.println("\nUpdated Temperature:");
        temp.display();

        // Test validation
        temp.setCelsius(-300);

        System.out.println("\nAfter Invalid Update:");
        temp.display();
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56308" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab03.TemperatureConverter

Original Temperature:

Celsius: 25.00°C

Fahrenheit: 77.00°F

Updated Temperature:

Celsius: 37.00°C

Fahrenheit: 98.60°F

Invalid temperature!

After Invalid Update:

Celsius: 37.00°C

Fahrenheit: 98.60°F

Process finished with exit code 0

TransactionAccount.java

```
package lab03;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class TransactionAccount {
    // Private fields
    private String accountHolder;
    private double balance;

    // Private transaction history
    private final List<String> transactions = new ArrayList<>();

    // Constructor
    public TransactionAccount(String accountHolder, double balance) {
        this.accountHolder = accountHolder;

        if (balance >= 0) {
            this.balance = balance;
        } else {
            this.balance = 0;
        }

        transactions.add("Opening balance: $" + this.balance);
    }

    // Deposit money
    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            transactions.add("Deposited: $" + amount);
            System.out.println("Deposit successful!");
        } else {
            System.out.println("Invalid deposit amount!");
        }
    }

    // Withdraw money
    public void withdraw(double amount) {
        if (amount <= 0) {
            System.out.println("Invalid withdrawal amount!");
        } else if (amount > balance) {
            System.out.println("Insufficient funds!");
```

```
        } else {
            balance -= amount;
            transactions.add("Withdrawn: $" + amount);
            System.out.println("Withdrawal successful!");
        }
    }

    // Getter for balance
    public double getBalance() {
        return balance;
    }

    // Return a read-only view of transaction history
    public List<String> getTransactions() {
        return Collections.unmodifiableList(transactions);
    }

    // Display account information
    public void display() {
        System.out.println("Account Holder: " + accountHolder);
        System.out.println("Balance: $" + balance);
    }

    // Display transaction history
    public void displayTransactions() {
        System.out.println("\nTransaction History:");

        for (String transaction : transactions) {
            System.out.println(transaction);
        }
    }

    public static void main(String[] args) {

        TransactionAccount account =
                new TransactionAccount("Precious", 500);

        account.deposit(200);
        account.withdraw(100);
        account.deposit(300);
        account.withdraw(50);

        account.display();
        account.displayTransactions();
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56311" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab03.TransactionAccount

Deposit successful!

Withdrawal successful!

Deposit successful!

Withdrawal successful!

Account Holder: Precious

Balance: $850.0

Transaction History:

Opening balance: $500.0

Deposited: $200.0

Withdrawn: $100.0

Deposited: $300.0

Withdrawn: $50.0

Process finished with exit code 0

6.5 Conclusion

Lab 3 showed that encapsulation improves data protection, reliability and maintainability. It is good practice to keep fields private and provide only the methods needed to interact with an object.

7. LAB 4: INHERITANCE

7.1 Objectives

The objectives of Lab 4 were to:

- Create subclasses using the `extends` keyword.
- Reuse superclass fields and methods.

- Call superclass constructors and methods using `super`.
- Understand the use of `protected` and `final`.
- Distinguish inheritance from composition.

7.2 Introduction

Inheritance allows one class to acquire accessible features from another class. The original class is known as the superclass, while the class that inherits from it is the subclass.

Inheritance supports code reuse and allows specialised classes to extend or modify existing behaviour. Composition, on the other hand, represents a relationship in which one object contains or uses another object.

7.3 Practical Work Performed

An employee hierarchy was used to demonstrate inheritance. The `Employee` class represented common employee information and behaviour. Subclasses such as `Manager` and `Developer` reused the employee functionality and provided specialised methods for calculating pay.

The `super` keyword was used to call the superclass constructor and reuse superclass behaviour. The `@Override` annotation helped identify methods intended to override inherited methods.

The exercises also illustrated the difference between an *is-a* relationship and a *has-a* relationship.

7.4 Results and Observations

The exercises showed that inheritance reduces duplication when classes share common characteristics. A subclass can reuse existing methods and introduce additional fields or specialised behaviour.

The superclass constructor runs before the subclass constructor completes its initialisation. The `final` modifier can prevent a class from being extended or a method from being overridden.

Animal.java

```
package lab04;

public class Animal {

    protected String name;

    public Animal(String name) {
        this.name = name;
    }

    public void eat() {
        System.out.println(name + " is eating.");
```

```
    }
}
```

AnimalSoundDemo.java

```
package lab04;

public class AnimalSoundDemo {

    // Parent class
    static class SoundAnimal {
        public void makeSound() {
            System.out.println("Animal makes a sound.");
        }
    }

    // Child class Dog
    static class SoundDog extends SoundAnimal {
        @Override
        public void makeSound() {
            System.out.println("Dog barks: Woof Woof!");
        }
    }

    // Child class Cat
    static class SoundCat extends SoundAnimal {
        @Override
        public void makeSound() {
            System.out.println("Cat meows: Meow Meow!");
        }
    }

    public static void main(String[] args) {

        SoundAnimal animal = new SoundAnimal();
        SoundDog dog = new SoundDog();
        SoundCat cat = new SoundCat();

        animal.makeSound();
        dog.makeSound();
        cat.makeSound();
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56313" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab04.AnimalSoundDemo

Animal makes a sound.

Dog barks: Woof Woof!

Cat meows: Meow Meow!

Process finished with exit code 0

Dog.java

```java
package lab04;

public class Dog extends Animal {

        public Dog(String name) {
            super(name);
        }

        public void bark() {
            System.out.println(name + " is barking.");
        }

        public static void main(String[] args) {

            Dog dog = new Dog("Buddy");

            dog.eat();
            dog.bark();
        }
    }
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56315" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab04.Dog

Buddy is eating.

Buddy is barking.

Process finished with exit code 0

EmployeeInheritanceDemo.java

```
package lab04;

public class EmployeeInheritanceDemo {

    // Parent class
    static class Employee {
        protected String name;
        protected double salary;

        public Employee(String name, double salary) {
            this.name = name;
            this.salary = salary;
        }

        public double calculateSalary() {
            return salary;
        }

        public void displayDetails() {
            System.out.println("Name: " + name);
            System.out.println("Salary: $" + calculateSalary());
        }
    }

    // Child class
    static class Manager extends Employee {
        private double bonus;

        public Manager(String name, double salary, double bonus) {
            super(name, salary);
            this.bonus = bonus;
        }

        // Override the parent's salary calculation
        @Override
        public double calculateSalary() {
            return super.calculateSalary() + bonus;
        }

        @Override
        public void displayDetails() {
            System.out.println("Position: Manager");
            super.displayDetails();
            System.out.println("Bonus: $" + bonus);
        }
    }

    public static void main(String[] args) {
```

```
        Employee employee = new Employee("Tendai", 800);
        Manager manager = new Manager("Precious", 1500, 300);

        System.out.println("Employee Details:");
        employee.displayDetails();

        System.out.println("\nManager Details:");
        manager.displayDetails();
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56317" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab04.EmployeeInheritanceDemo

Employee Details:

Name: Tendai

Salary: $800.0

Manager Details:

Position: Manager

Name: Precious

Salary: $1800.0

Bonus: $300.0

Process finished with exit code 0

MultilevelInheritanceDemo.java

```
package lab04;

public class MultilevelInheritanceDemo {

    // Grandparent class
```

```
    static class Vehicle {

        public void start() {
            System.out.println("Vehicle is starting.");
        }
    }

    // Child class inherits from Vehicle
    static class Car extends Vehicle {

        public void drive() {
            System.out.println("Car is driving.");
        }
    }

    // Grandchild class inherits from Car
    static class ElectricCar extends Car {

        public void charge() {
            System.out.println("Electric car is charging.");
        }
    }

    public static void main(String[] args) {

        // Create an ElectricCar object
        ElectricCar myCar = new ElectricCar();

        // Method inherited from Vehicle
        myCar.start();

        // Method inherited from Car
        myCar.drive();

        // Method defined in ElectricCar
        myCar.charge();
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56319" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab04.MultilevelInheritanceDemo

Vehicle is starting.

Car is driving.

Electric car is charging.

Process finished with exit code 0

SuperKeywordDemo.java

```
package lab04;

public class SuperKeywordDemo {

    // Parent class
    static class Person {

        protected String name;

        // Parent constructor
        public Person(String name) {
            this.name = name;
            System.out.println("Person constructor called.");
        }

        public void display() {
            System.out.println("Name: " + name);
        }
    }

    // Child class
    static class Student extends Person {

        private String registrationNumber;

        // Child constructor
        public Student(String name, String registrationNumber) {

            // Call parent constructor
            super(name);

            this.registrationNumber = registrationNumber;
            System.out.println("Student constructor called.");
        }

        // Override parent method
        @Override
        public void display() {

            // Call parent's display method
            super.display();

            System.out.println(
                    "Registration Number: " + registrationNumber
            );
        }
    }

    public static void main(String[] args) {
```

```
        Student student = new Student(
                "Precious Millicent Zvidi",
                "H250137J"
        );

        System.out.println("\nStudent Details:");
        student.display();
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56321" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab04.SuperKeywordDemo

Person constructor called.

Student constructor called.

Student Details:

Name: Precious Millicent Zvidi

Registration Number: H250137J

Process finished with exit code 0

7.5 Conclusion

Lab 4 demonstrated how inheritance supports reuse and specialisation in Java. It also highlighted the importance of choosing inheritance only when a genuine relationship exists between the classes.

8. LAB 5: POLYMORPHISM

8.1 Objectives

The objectives of Lab 5 were to:

- Understand method overloading and method overriding.
- Apply runtime polymorphism.
- Use superclass references to refer to subclass objects.
- Understand dynamic method dispatch.
- Use type checking and casting appropriately.

8.2 Introduction

Polymorphism means that one interface or general type can be used to represent different forms of behaviour. In Java, method overloading and method overriding are two important ways of achieving polymorphic behaviour.

Overloading occurs when methods have the same name but different parameter lists. Overriding occurs when a subclass provides its own implementation of an inherited instance method with a compatible signature.

8.3 Practical Work Performed

Calculator-style examples were used to demonstrate overloaded methods that performed similar operations with different parameter lists.

Inheritance examples demonstrated overriding, in which subclasses provided their own versions of methods defined in a superclass. A superclass reference could refer to a subclass object, allowing the overridden method to be selected according to the object's actual type at runtime.

Casting exercises demonstrated the importance of checking an object's type before attempting a downcast.

8.4 Results and Observations

The exercises demonstrated that overloading is resolved by the compiler based on the method arguments. Overridden instance methods are selected at runtime based on the actual object.

Polymorphism allows a program to work with a general type while still using the specialised behaviour of different subclasses.

Calculator.java

```
package lab05;

public class Calculator {

    // Method 1: Add two integers
    public int add(int a, int b) {
        return a + b;
    }
```

```
    // Method 2: Add three integers
    public int add(int a, int b, int c) {
        return a + b + c;
    }

    // Method 3: Add two decimal numbers
    public double add(double a, double b) {
        return a + b;
    }

    public static void main(String[] args) {

        Calculator calc = new Calculator();

        System.out.println("Adding two integers: "
                + calc.add(10, 20));

        System.out.println("Adding three integers: "
                + calc.add(10, 20, 30));

        System.out.println("Adding two decimals: "
                + calc.add(5.5, 4.5));
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56390" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab05.Calculator

Adding two integers: 30

Adding three integers: 60

Adding two decimals: 10.0

Process finished with exit code 0

DowncastingDemo.java

```
package lab05;

public class DowncastingDemo {

    // Parent class
```

```java
    static class Animal {
        public void eat() {
            System.out.println("Animal is eating.");
        }
    }

    // Child class Dog
    static class Dog extends Animal {
        public void bark() {
            System.out.println("Dog is barking.");
        }
    }

    // Child class Cat
    static class Cat extends Animal {
        public void meow() {
            System.out.println("Cat is meowing.");
        }
    }

    public static void main(String[] args) {

        // Upcasting: child objects stored as Animal references
        Animal animal1 = new Dog();
        Animal animal2 = new Cat();

        animal1.eat();
        animal2.eat();

        // Check type before downcasting
        if (animal1 instanceof Dog) {
            Dog dog = (Dog) animal1;
            dog.bark();
        }

        if (animal2 instanceof Cat) {
            Cat cat = (Cat) animal2;
            cat.meow();
        }

        // Prevent an invalid cast
        if (animal1 instanceof Cat) {
            Cat cat = (Cat) animal1;
            cat.meow();
        } else {
            System.out.println(
                    "animal1 is not a Cat. Casting avoided."
            );
        }
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56395" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab05.DowncastingDemo

Animal is eating.

Animal is eating.

Dog is barking.

Cat is meowing.

animal1 is not a Cat. Casting avoided.

Process finished with exit code 0

EmployeePolymorphismDemo.java

```
package lab05;

public class EmployeePolymorphismDemo {

    // Parent class
    static class Employee {
        protected String name;

        public Employee(String name) {
            this.name = name;
        }

        public double calculateSalary() {
            return 0;
        }

        public void displayDetails() {
            System.out.println("Employee: " + name);
            System.out.println("Salary: $" + calculateSalary());
        }
    }

    // Full-time employee
    static class FullTimeEmployee extends Employee {
        private double monthlySalary;
```

```
        public FullTimeEmployee(String name, double monthlySalary) {
            super(name);
            this.monthlySalary = monthlySalary;
        }

        @Override
        public double calculateSalary() {
            return monthlySalary;
        }
    }

    // Part-time employee
    static class PartTimeEmployee extends Employee {
        private double hourlyRate;
        private int hoursWorked;

        public PartTimeEmployee(String name,
                                double hourlyRate,
                                int hoursWorked) {
            super(name);
            this.hourlyRate = hourlyRate;
            this.hoursWorked = hoursWorked;
        }

        @Override
        public double calculateSalary() {
            return hourlyRate * hoursWorked;
        }
    }

    public static void main(String[] args) {

        // One array stores different employee types
        Employee[] employees = {
                new FullTimeEmployee("Precious", 1500),
                new PartTimeEmployee("Tendai", 10, 80),
                new FullTimeEmployee("Rudo", 1200),
                new PartTimeEmployee("Tariro", 15, 60)
        };

        // Process all employees using one loop
        for (Employee employee : employees) {
            employee.displayDetails();
            System.out.println("--------------------");
        }
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56410" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab05.EmployeePolymorphismDemo

Employee: Precious

Salary: $1500.0

Employee: Tendai

Salary: $800.0

Employee: Rudo

Salary: $1200.0

Employee: Tariro

Salary: $900.0

Process finished with exit code 0

RutimePolymorphismDemo.java

```
package lab05;

public class RuntimePolymorphismDemo {

    // Parent class
    static class Animal {

        public void makeSound() {
            System.out.println("Animal makes a sound.");
        }
    }

    // Child class Dog
    static class Dog extends Animal {

        @Override
        public void makeSound() {
            System.out.println("Dog says: Woof Woof!");
        }
    }
```

```
    // Child class Cat
    static class Cat extends Animal {

        @Override
        public void makeSound() {
            System.out.println("Cat says: Meow Meow!");
        }
    }

    public static void main(String[] args) {

        // Parent reference pointing to a Dog object
        Animal animal1 = new Dog();

        // Parent reference pointing to a Cat object
        Animal animal2 = new Cat();

        // Java selects the appropriate method at runtime
        animal1.makeSound();
        animal2.makeSound();
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56439" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab05.RuntimePolymorphismDemo

Dog says: Woof Woof!

Cat says: Meow Meow!

Process finished with exit code 0

ShapePolymorphismDemo.java

```
package lab05;

public class ShapePolymorphismDemo {


    // Parent class
    static class Shape {

        public double calculateArea() {
            return 0;
        }
```

```
    public void displayArea() {
        System.out.printf("Area: %.2f%n", calculateArea());
    }
}

// Child class: Circle
static class Circle extends Shape {

    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    @Override
    public double calculateArea() {
        return Math.PI * radius * radius;
    }
}

// Child class: Rectangle
static class Rectangle extends Shape {

    private double length;
    private double width;

    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    @Override
    public double calculateArea() {
        return length * width;
    }
}

// Child class: Triangle
static class Triangle extends Shape {

    private double base;
    private double height;

    public Triangle(double base, double height) {
        this.base = base;
        this.height = height;
    }

    @Override
    public double calculateArea() {
        return 0.5 * base * height;
    }
}

public static void main(String[] args) {
```

```
        // Polymorphic array
        Shape[] shapes = {
                new Circle(5),
                new Rectangle(4, 6),
                new Triangle(8, 3)
        };

        String[] shapeNames = {
                "Circle", "Rectangle", "Triangle"
        };

        // Process all shapes using one loop
        for (int i = 0; i < shapes.length; i++) {
            System.out.println("Shape: " + shapeNames[i]);
            shapes[i].displayArea();
            System.out.println("--------------------");
        }
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56447" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab05.ShapePolymorphismDemo

Shape: Circle

Area: 78.54

Shape: Rectangle

Area: 24.00

Shape: Triangle

Area: 12.00

Process finished with exit code 0

8.5 Conclusion

Lab 5 demonstrated that polymorphism improves flexibility and extensibility by allowing different objects to respond to the same method call in different ways.

9. LAB 6: ABSTRACTION AND INTERFACES

9.1 Objectives

The objectives of Lab 6 were to:

- Create abstract classes and abstract methods.
- Implement interfaces.
- Distinguish between abstract classes and interfaces.
- Apply abstraction in class design.
- Program against common contracts.

9.2 Introduction

Abstraction focuses on essential features while hiding unnecessary implementation details. An abstract class can contain fields, constructors, implemented methods and abstract methods that subclasses must implement.

An interface defines a contract that implementing classes agree to follow. Interfaces are useful when different classes need to provide similar capabilities even if they do not share the same parent class.

9.3 Practical Work Performed

The exercises used abstract classes to define common behaviour and abstract methods for subclasses to implement. Interfaces were used to describe behaviours that could be shared by different classes.

Implementing classes supplied the required method bodies. These examples demonstrated how code could depend on an abstract type or interface rather than on one specific implementation.

9.4 Results and Observations

The exercises showed that an abstract class cannot be instantiated directly when it is declared abstract. Concrete subclasses must implement its remaining abstract methods before they can be instantiated.

Interfaces help standardise behaviour and make it easier to replace one implementation with another. Abstract classes are particularly useful when related classes need to share state or implemented functionality.

AbstractShapeDemo.java

```
package lab06;

public class AbstractShapeDemo {

    // Abstract parent class
    static abstract class Shape {

        // Abstract method without a body
        public abstract double calculateArea();

        // Concrete method with a body
        public void displayArea() {
            System.out.printf("Area: %.2f%n", calculateArea());
        }
    }

    // Circle subclass
    static class Circle extends Shape {

        private double radius;

        public Circle(double radius) {
            this.radius = radius;
        }

        @Override
        public double calculateArea() {
            return Math.PI * radius * radius;
        }
    }

    // Rectangle subclass
    static class Rectangle extends Shape {

        private double length;
        private double width;

        public Rectangle(double length, double width) {
            this.length = length;
            this.width = width;
        }

        @Override
        public double calculateArea() {
```

```
            return length * width;
        }
    }

    public static void main(String[] args) {

        // Abstract classes cannot be instantiated directly
        // Shape shape = new Shape();

        Shape circle = new Circle(5);
        Shape rectangle = new Rectangle(4, 6);

        System.out.println("Circle:");
        circle.displayArea();

        System.out.println("Rectangle:");
        rectangle.displayArea();
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56459" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab06.AbstractShapeDemo

Circle:

Area: 78.54

Rectangle:

Area: 24.00

Process finished with exit code 0

DefaultMethodDemo.java

```
package lab06;

public class DefaultMethodDemo {

    // Interface containing an abstract and a default method
    interface Vehicle {

        // Abstract method
```

```
        void start();

        // Default method with an implementation
        default void stop() {
            System.out.println("Vehicle is stopping.");
        }
    }

    // Car uses the default stop() method
    static class Car implements Vehicle {

        @Override
        public void start() {
            System.out.println("Car is starting.");
        }
    }

    // Motorcycle overrides the default stop() method
    static class Motorcycle implements Vehicle {

        @Override
        public void start() {
            System.out.println("Motorcycle is starting.");
        }

        @Override
        public void stop() {
            System.out.println("Motorcycle is stopping safely.");
        }
    }

    public static void main(String[] args) {

        Vehicle car = new Car();
        Vehicle motorcycle = new Motorcycle();

        System.out.println("Car:");
        car.start();
        car.stop();

        System.out.println("\nMotorcycle:");
        motorcycle.start();
        motorcycle.stop();
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56477" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab06.DefaultMethodDemo

Car:

Car is starting.

Vehicle is stopping.

Motorcycle:

Motorcycle is starting.

Motorcycle is stopping safely.

Process finished with exit code 0

EmployeePaymentDemo.java

```
package lab06;

public class EmployeePaymentDemo {

    // Interface defining a payment contract
    interface Payable {
        double calculatePayment();
    }

    // Abstract class implementing the interface
    static abstract class Employee implements Payable {

        private String name;

        public Employee(String name) {
            this.name = name;
        }

        public String getName() {
            return name;
        }

        // Abstract method to be implemented by subclasses
        public abstract double calculatePayment();

        // Concrete method shared by all employees
        public void displayPayment() {
            System.out.println("Employee: " + getName());
            System.out.printf("Payment: $%.2f%n",
                    calculatePayment());
        }
    }
```

```java
    // Full-time employee
    static class FullTimeEmployee extends Employee {

        private double monthlySalary;

        public FullTimeEmployee(String name, double monthlySalary) {
            super(name);
            this.monthlySalary = monthlySalary;
        }

        @Override
        public double calculatePayment() {
            return monthlySalary;
        }
    }

    // Part-time employee
    static class PartTimeEmployee extends Employee {

        private double hourlyRate;
        private int hoursWorked;

        public PartTimeEmployee(String name,
                                double hourlyRate,
                                int hoursWorked) {
            super(name);
            this.hourlyRate = hourlyRate;
            this.hoursWorked = hoursWorked;
        }

        @Override
        public double calculatePayment() {
            return hourlyRate * hoursWorked;
        }
    }

    public static void main(String[] args) {

        // Array of abstract-class references
        Employee[] employees = {
                new FullTimeEmployee("Precious", 1500),
                new PartTimeEmployee("Tendai", 12, 80),
                new FullTimeEmployee("Rudo", 1200)
        };

        // Process employees using polymorphism
        for (Employee employee : employees) {
            employee.displayPayment();
            System.out.println("--------------------");
        }
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56483" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab06.EmployeePaymentDemo

Employee: Precious

Payment: $1500.00

Employee: Tendai

Payment: $960.00

Employee: Rudo

Payment: $1200.00

Process finished with exit code 0

MultipleInterfacesDemo.java

```
package lab06;

public class MultipleInterfacesDemo {

    // First interface
    interface Camera {
        void takePhoto();
    }

    // Second interface
    interface MusicPlayer {
        void playMusic();
    }

    // Smartphone implements both interfaces
    static class Smartphone implements Camera, MusicPlayer {

        @Override
        public void takePhoto() {
            System.out.println("Smartphone is taking a photo.");
        }
```

```
        @Override
        public void playMusic() {
            System.out.println("Smartphone is playing music.");
        }

        public void makeCall() {
            System.out.println("Smartphone is making a call.");
        }
    }

    public static void main(String[] args) {

        Smartphone phone = new Smartphone();

        // Methods from the two interfaces
        phone.takePhoto();
        phone.playMusic();

        // Method belonging to Smartphone
        phone.makeCall();

        // Interface references
        Camera camera = phone;
        MusicPlayer player = phone;

        System.out.println("\nUsing interface references:");
        camera.takePhoto();
        player.playMusic();
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56490" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab06.MultipleInterfacesDemo

Smartphone is taking a photo.

Smartphone is playing music.

Smartphone is making a call.

Using interface references:

Smartphone is taking a photo.

Smartphone is playing music.

Process finished with exit code 0

PaymentInterfaceDemo.java

```
package lab06;

public class PaymentInterfaceDemo {

    // Interface defining a payment contract
    interface Payment {
        void pay(double amount);
    }

    // First implementation: Cash payment
    static class CashPayment implements Payment {

        @Override
        public void pay(double amount) {
            System.out.println(
                    "Cash payment of $" + amount + " completed."
            );
        }
    }

    // Second implementation: Card payment
    static class CardPayment implements Payment {

        @Override
        public void pay(double amount) {
            System.out.println(
                    "Card payment of $" + amount + " completed."
            );
        }
    }

    // Third implementation: Mobile payment
    static class MobilePayment implements Payment {

        @Override
        public void pay(double amount) {
            System.out.println(
                    "Mobile payment of $" + amount + " completed."
            );
        }
    }

    public static void main(String[] args) {

        // Interface references holding different implementations
        Payment payment1 = new CashPayment();
        Payment payment2 = new CardPayment();
```

```
        Payment payment3 = new MobilePayment();

        payment1.pay(100);
        payment2.pay(250);
        payment3.pay(75);
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56498" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab06.PaymentInterfaceDemo

Cash payment of $100.0 completed.

Card payment of $250.0 completed.

Mobile payment of $75.0 completed.

Process finished with exit code 0

9.5 Conclusion

Lab 6 demonstrated how abstraction reduces complexity and encourages reusable, flexible program designs through abstract classes and interfaces.

10. LAB 7: EXCEPTION HANDLING

10.1 Objectives

The objectives of Lab 7 were to:

- Handle exceptions using `try`, `catch` and `finally`.
- Understand checked and unchecked exceptions.
- Create custom exceptions.
- Use try-with-resources.
- Improve program reliability when errors occur.

10.2 Introduction

An exception is an event that interrupts the normal execution of a program. Java provides exception-handling mechanisms that allow programs to respond to errors instead of terminating unexpectedly.

The `try` block contains code that may cause an exception. A `catch` block handles a matching exception, while a `finally` block can be used for cleanup. Try-with-resources automatically closes supported resources.

10.3 Practical Work Performed

The practical exercises demonstrated how potentially problematic operations could be placed inside `try` blocks. Catch blocks were used to handle specific exceptions and display appropriate messages.

Custom exceptions were used to represent application-specific errors. Try-with-resources demonstrated a safer way of managing resources such as files that implement `AutoCloseable`.

10.4 Results and Observations

The exercises showed that exceptions can be handled at appropriate points in a program. Using specific exception types makes error handling clearer and avoids treating every error in the same way.

Try-with-resources automatically closes supported resources when execution leaves the statement, including when an exception occurs.

ArithmeticExceptionDemo.java

```
package lab07;

public class ArithmeticExceptionDemo {

    public static void main(String[] args) {

        int numerator = 20;
        int denominator = 0;

        System.out.println("Program started.");

        try {
            // This statement causes an ArithmeticException
            int result = numerator / denominator;

            System.out.println("Result: " + result);

        } catch (ArithmeticException e) {

            // Handle division by zero
            System.out.println("Error: Cannot divide by zero.");
```

```
            System.out.println("Exception: " + e.getMessage());
        }

        // Program continues after handling the exception
        System.out.println("Program continues normally.");
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56506" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab07.ArithmeticExceptionDemo

Program started.

Error: Cannot divide by zero.

Exception: / by zero

Program continues normally.

Process finished with exit code 0

CustomExceptionDemo.java

```
package lab07;

public class CustomExceptionDemo {

    // Custom checked exception
    static class InsufficientBalanceException extends Exception {

        public InsufficientBalanceException(String message) {
            super(message);
        }
    }

    // Bank account class
    static class BankAccount {

        private double balance;

        public BankAccount(double balance) {
            this.balance = balance;
        }
```

```
        public void withdraw(double amount)
                throws InsufficientBalanceException {

            if (amount <= 0) {
                throw new IllegalArgumentException(
                        "Withdrawal amount must be positive."
                );
            }

            if (amount > balance) {
                throw new InsufficientBalanceException(
                        "Insufficient balance! Available balance: $" +
balance
                );
            }

            balance -= amount;

            System.out.println("Withdrawal successful.");
            System.out.println("Amount withdrawn: $" + amount);
            System.out.println("Remaining balance: $" + balance);
        }
    }

    public static void main(String[] args) {

        BankAccount account = new BankAccount(500);

        try {

            account.withdraw(700);

        } catch (InsufficientBalanceException e) {

            System.out.println("Custom Exception: " + e.getMessage());

        } catch (IllegalArgumentException e) {

            System.out.println("Invalid Amount: " + e.getMessage());
        }

        System.out.println("Banking operation completed.");
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56510" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab07.CustomExceptionDemo

Custom Exception: Insufficient balance! Available balance: $500.0

Banking operation completed.

Process finished with exit code 0

FinallyBlockDemo.java

```
package lab07;

public class FinallyBlockDemo {

    public static void main(String[] args) {

        System.out.println("Program started.");

        try {

            int numerator = 20;
            int denominator = 0;

            int result = numerator / denominator;

            System.out.println("Result: " + result);

        } catch (ArithmeticException e) {

            System.out.println("Error: Cannot divide by zero.");

        } finally {

            System.out.println("Finally block executed.");
            System.out.println("Cleanup completed.");

        }

        System.out.println("Program ended.");
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56528" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab07.FinallyBlockDemo

Program started.

Error: Cannot divide by zero.

Finally block executed.

Cleanup completed.

Program ended.

Process finished with exit code 0

MultipleCatchDemo.java

```
package lab07;

public class MultipleCatchDemo {

    public static void main(String[] args) {

        int[] numbers = {10, 20, 30};

        try {

            // Change this value to test different exceptions
            int choice = 1;

            if (choice == 1) {

                // Arithmetic exception
                int result = 10 / 0;
                System.out.println(result);

            } else if (choice == 2) {

                // Array index exception
                System.out.println(numbers[5]);

            } else if (choice == 3) {

                // Number format exception
                int value = Integer.parseInt("abc");
                System.out.println(value);

            } else {

                System.out.println("No exception occurred.");
            }

        } catch (ArithmeticException e) {

            System.out.println("Arithmetic Error: Division by zero.");
```

```
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Array Error: Invalid array index.");
        } catch (NumberFormatException e) {
            System.out.println("Number Format Error: Invalid number.");
        }
        System.out.println("Program completed.");
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56539" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab07.MultipleCatchDemo

Arithmetic Error: Division by zero.

Program completed.

Process finished with exit code 0

ThrowThrowsDemo.java

```
package lab07;

public class ThrowThrowsDemo {

    // Method declaring that it may throw an exception
    public static void withdraw(double balance, double amount)
            throws Exception {

        if (amount <= 0) {
            throw new Exception(
                    "Withdrawal amount must be greater than zero."
            );
        }

        if (amount > balance) {
            throw new Exception(
                    "Insufficient funds. Withdrawal denied."
            );
```

```
        }

        double remainingBalance = balance - amount;

        System.out.println("Withdrawal successful.");
        System.out.println("Amount withdrawn: $" + amount);
        System.out.println("Remaining balance: $" + remainingBalance);
    }

    public static void main(String[] args) {

        double balance = 500;
        double amount = 700;

        try {

            withdraw(balance, amount);

        } catch (Exception e) {

            System.out.println("Error: " + e.getMessage());

        }

        System.out.println("Transaction process completed.");
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56547" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab07.ThrowThrowsDemo

Error: Insufficient funds. Withdrawal denied.

Transaction process completed.

Process finished with exit code 0

10.5 Conclusion

Lab 7 demonstrated that exception handling is important for creating robust Java applications. It improves error reporting and helps ensure that resources are managed correctly.

11. LAB 8: GENERICS AND COLLECTIONS

11.1 Objectives

The objectives of Lab 8 were to:

- Create generic classes and methods.
- Understand and use `List`, `Set` and `Map`.
- Store and retrieve collections of objects.
- Sort data using a `Comparator`.
- Apply type safety when handling data.

11.2 Introduction

Generics allow classes, interfaces and methods to work with specified types while maintaining compile-time type checking. Collections are structures used to store and manage groups of objects.

A `List` stores elements in a sequence and can contain duplicates. A `Set` represents unique elements, while a `Map` stores values associated with keys. A `Comparator` defines a rule for comparing objects when sorting them.

11.3 Practical Work Performed

The exercises demonstrated the creation of generic classes and methods and the storage of data in collections. Lists were used for ordered data, sets for unique values, and maps for key-value relationships.

Sorting exercises demonstrated how comparators can arrange objects according to selected fields, such as names or numerical values.

11.4 Results and Observations

The practical work showed that generics reduce the need for unsafe type casting and help detect type errors during compilation.

Different collection types serve different purposes. Selecting an appropriate collection improves the clarity and efficiency of a program.

ArrayListDemo.java

```java
package lab08;

import java.util.HashSet;

public class HashSetDemo {

    public static void main(String[] args) {

        // Create a HashSet for student registration numbers
        HashSet<String> students = new HashSet<>();

        // Add registration numbers
        students.add("H250137J");
        students.add("H250138K");
        students.add("H250139L");
        students.add("H250140M");

        // Attempt to add a duplicate
        boolean added = students.add("H250137J");

        System.out.println("Original Student Records:");
        System.out.println(students);

        // Check whether duplicate was added
        System.out.println("\nDuplicate added? " + added);

        // Check whether a student exists
        System.out.println("\nDoes H250137J exist?");
        System.out.println(students.contains("H250137J"));

        // Remove a student
        students.remove("H250139L");

        System.out.println("\nAfter removing H250139L:");
        System.out.println(students);

        // Display number of unique students
        System.out.println("\nTotal unique students: "
                + students.size());

        // Loop through the HashSet
        System.out.println("\nStudent Registration Numbers:");

        for (String student : students) {
            System.out.println(student);
        }
    }
}
```

```
"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56569" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -
```

Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab08.ArrayListDemo

Original student list:

[Precious, Tendai, Rudo, Tapiwa]

First student: Precious

After updating:

[Precious, Brian, Rudo, Tapiwa]

After removing Rudo:

[Precious, Brian, Tapiwa]

Number of students: 3

Student names:

Precious

Brian

Tapiwa

Process finished with exit code 0

GenericsClassDemo.java

```
package lab08;

public class GenericClassDemo {

    // Generic class that works with different data types
```

```java
    static class Box<T> {

        private T value;

        // Constructor
        public Box(T value) {
            this.value = value;
        }

        // Getter method
        public T getValue() {
            return value;
        }

        // Setter method
        public void setValue(T value) {
            this.value = value;
        }

        // Display the stored value
        public void display() {
            System.out.println("Stored value: " + value);
        }
    }

    public static void main(String[] args) {

        // Box storing an Integer
        Box<Integer> integerBox = new Box<>(100);

        // Box storing a String
        Box<String> stringBox = new Box<>("Hello Java");

        // Box storing a Double
        Box<Double> doubleBox = new Box<>(45.75);

        System.out.println("Integer Box:");
        integerBox.display();

        System.out.println("\nString Box:");
        stringBox.display();

        System.out.println("\nDouble Box:");
        doubleBox.display();

        // Updating a value using the setter
        integerBox.setValue(200);

        System.out.println("\nUpdated Integer Box:");
        System.out.println("New value: " + integerBox.getValue());
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56577" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab08.GenericClassDemo

Integer Box:

Stored value: 100

String Box:

Stored value: Hello Java

Double Box:

Stored value: 45.75

Updated Integer Box:

New value: 200

Process finished with exit code 0

GenericMethodDemo.java

```java
package lab08;

public class GenericMethodDemo {

    // Generic method to display an array
    public static <T> void displayArray(T[] array) {

        for (T element : array) {
            System.out.println(element);
        }
    }

    // Generic method to return the first element
    public static <T> T getFirstElement(T[] array) {

        if (array.length == 0) {
```

```
            throw new IllegalArgumentException(
                    "Array cannot be empty."
            );
        }

        return array[0];
    }

    public static void main(String[] args) {

        // Integer array
        Integer[] numbers = {10, 20, 30, 40};

        // String array
        String[] names = {"Precious", "Tendai", "Rudo"};

        // Double array
        Double[] prices = {12.50, 25.75, 40.00};

        System.out.println("Integer Array:");
        displayArray(numbers);

        System.out.println("\nString Array:");
        displayArray(names);

        System.out.println("\nDouble Array:");
        displayArray(prices);

        // Using a generic method with different types
        System.out.println("\nFirst Integer: "
                + getFirstElement(numbers));

        System.out.println("First Name: "
                + getFirstElement(names));

        System.out.println("First Price: "
                + getFirstElement(prices));
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56585" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab08.GenericMethodDemo

Integer Array:

10

20

30

40

String Array:

Precious

Tendai

Rudo

Double Array:

12.5

25.75

40.0

First Integer: 10

First Name: Precious

First Price: 12.5

Process finished with exit code 0

HashMapDemo.java

```
package lab08;

import java.util.HashMap;

public class HashMapDemo {

    public static void main(String[] args) {

        // Create a HashMap to store student details
        HashMap<String, String> students = new HashMap<>();
```

```
        // Add student registration numbers and names
        students.put("H250137J", "Precious");
        students.put("H250138K", "Tendai");
        students.put("H250139L", "Rudo");
        students.put("H250140M", "Brian");

        // Display all students
        System.out.println("Student Records:");
        System.out.println(students);

        // Retrieve a student using registration number
        System.out.println("\nStudent H250137J:");
        System.out.println(students.get("H250137J"));

        // Update a student's name
        students.put("H250138K", "Tapiwa");

        System.out.println("\nAfter updating:");
        System.out.println(students);

        // Remove a student
        students.remove("H250139L");

        System.out.println("\nAfter removing a student:");
        System.out.println(students);

        // Check if a registration number exists
        System.out.println("\nDoes H250137J exist?");
        System.out.println(students.containsKey("H250137J"));

        // Display the total number of students
        System.out.println("\nTotal students: " + students.size());

        // Loop through all student records
        System.out.println("\nStudent Registration Records:");

        for (String regNumber : students.keySet()) {
            System.out.println(
                    regNumber + " : " + students.get(regNumber)
            );
        }
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56594" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab08.HashMapDemo

Student Records:

{H250137J=Precious, H250138K=Tendai, H250139L=Rudo, H250140M=Brian}

Student H250137J:

Precious

After updating:

{H250137J=Precious, H250138K=Tapiwa, H250139L=Rudo, H250140M=Brian}

After removing a student:

{H250137J=Precious, H250138K=Tapiwa, H250140M=Brian}

Does H250137J exist?

true

Total students: 3

Student Registration Records:

H250137J : Precious

H250138K : Tapiwa

H250140M : Brian

Process finished with exit code 0

HashSetDemo.java

```
package lab08;

import java.util.HashSet;

public class HashSetDemo {
```

```java
    public static void main(String[] args) {

        // Create a HashSet for student registration numbers
        HashSet<String> students = new HashSet<>();

        // Add registration numbers
        students.add("H250137J");
        students.add("H250138K");
        students.add("H250139L");
        students.add("H250140M");

        // Attempt to add a duplicate
        boolean added = students.add("H250137J");

        System.out.println("Original Student Records:");
        System.out.println(students);

        // Check whether duplicate was added
        System.out.println("\nDuplicate added? " + added);

        // Check whether a student exists
        System.out.println("\nDoes H250137J exist?");
        System.out.println(students.contains("H250137J"));

        // Remove a student
        students.remove("H250139L");

        System.out.println("\nAfter removing H250139L:");
        System.out.println(students);

        // Display number of unique students
        System.out.println("\nTotal unique students: "
                + students.size());

        // Loop through the HashSet
        System.out.println("\nStudent Registration Numbers:");

        for (String student : students) {
            System.out.println(student);
        }
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56598" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab08.HashSetDemo

Original Student Records:

[H250137J, H250138K, H250139L, H250140M]

Duplicate added? false

Does H250137J exist?

true

After removing H250139L:

[H250137J, H250138K, H250140M]

Total unique students: 3

Student Registration Numbers:

H250137J

H250138K

H250140M

Process finished with exit code 0

11.5 Conclusion

Lab 8 demonstrated how generics and collections make Java programs safer and more effective when managing groups of objects.

12. LAB 9: NESTED TYPES, ENUMS AND RECORDS

12.1 Objectives

The objectives of Lab 9 were to:

- Understand nested and inner classes.
- Represent fixed sets of values using enums.
- Use records to represent data.
- Implement `equals()`, `hashCode()` and `toString()`.
- Understand logical equality between objects.

12.2 Introduction

Java supports nested classes, which are declared inside other classes. Enums represent a fixed set of named constants, while records provide a concise way to declare classes primarily intended to carry data.

The methods `equals()`, `hashCode()` and `toString()` are inherited from the `Object` class and can be overridden to define meaningful equality, hash values and text representations.

12.3 Practical Work Performed

The practical exercises explored nested types and the use of inner classes. Enums were used to represent a fixed set of options rather than relying on arbitrary strings.

Records demonstrated a concise approach to storing data. Object-method exercises illustrated how to display objects in a readable form and how to compare their contents logically.

12.4 Results and Observations

The exercises showed that enums make fixed choices clearer and safer. Records reduce repetitive code for data-focused classes.

When `equals()` is overridden, `hashCode()` should also be overridden consistently. If two objects are considered equal, they must return the same hash code so that hash-based collections such as `HashSet` and `HashMap` work correctly.

AnonymousInnerClassDemo.java

```
package lab09;

public class AnonymousInnerClassDemo {

    // Interface defining a greeting method
```

```java
    interface Greeting {
        void greet();
    }

    // Abstract class representing a student
    static abstract class Student {

        public abstract void displayDetails();
    }

    public static void main(String[] args) {

        // Anonymous class implementing an interface
        Greeting greeting = new Greeting() {

            @Override
            public void greet() {
                System.out.println(
                        "Welcome to Harare Institute of Technology!"
                );
            }
        };

        greeting.greet();

        // Anonymous class extending an abstract class
        Student student = new Student() {

            @Override
            public void displayDetails() {
                System.out.println("\nStudent Details:");
                System.out.println("Name: Precious Millicent Zvidi");
                System.out.println("Registration Number: H250137J");
                System.out.println("Department: Software Engineering");
            }
        };

        student.displayDetails();

        System.out.println("\nProgram completed successfully.");
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56290" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab09.AnonymousInnerClassDemo

Welcome to Harare Institute of Technology!

Student Details:

Name: Precious Millicent Zvidi

Registration Number: H250137J

Department: Software Engineering

Program completed successfully.

Process finished with exit code 0

EnumDemo.java

```
package lab09;

public class EnumDemo {

    // Enum representing student registration status
    enum StudentStatus {
        REGISTERED,
        PENDING,
        SUSPENDED,
        GRADUATED
    }

    // Student class
    static class Student {

        private String name;
        private StudentStatus status;

        public Student(String name, StudentStatus status) {
            this.name = name;
            this.status = status;
        }

        public void displayDetails() {

            System.out.println("Student Name: " + name);
            System.out.println("Registration Status: " + status);

            // Using switch with an enum
            switch (status) {

                case REGISTERED:
                    System.out.println(
                            "Student is fully registered."
                    );
                    break;
```

```
                case PENDING:
                    System.out.println(
                            "Student registration is pending."
                    );
                    break;

                case SUSPENDED:
                    System.out.println(
                            "Student is currently suspended."
                    );
                    break;

                case GRADUATED:
                    System.out.println(
                            "Student has completed studies."
                    );
                    break;
            }
        }
    }

    public static void main(String[] args) {

        Student student1 = new Student(
                "Precious",
                StudentStatus.REGISTERED
        );

        Student student2 = new Student(
                "Tendai",
                StudentStatus.PENDING
        );

        System.out.println("Student 1:");
        student1.displayDetails();

        System.out.println("\nStudent 2:");
        student2.displayDetails();
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56288" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab09.EnumDemo

Student 1:

Student Name: Precious

Registration Status: REGISTERED

Student is fully registered.

Student 2:

Student Name: Tendai

Registration Status: PENDING

Student registration is pending.

Process finished with exit code 0

InnerClassDemo.java

```
package lab09;

public class InnerClassDemo {

    // Outer class
    static class University {

        private String universityName;

        public University(String universityName) {
            this.universityName = universityName;
        }

        // Non-static inner class
        class Student {

            private String studentName;
            private String registrationNumber;

            public Student(String studentName,
                           String registrationNumber) {
                this.studentName = studentName;
                this.registrationNumber = registrationNumber;
            }

            public void displayDetails() {
                System.out.println("University: " + universityName);
                System.out.println("Student Name: " + studentName);
                System.out.println("Registration Number: "
                        + registrationNumber);
            }
        }
    }
```

```
    public static void main(String[] args) {

        // Create an outer class object
        University university =
                new University("Harare Institute of Technology");

        // Create an inner class object
        University.Student student =
                university.new Student(
                        "Precious Millicent Zvidi",
                        "H250137J"
                );

        // Display student details
        student.displayDetails();
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56286" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab09.InnerClassDemo

University: Harare Institute of Technology

Student Name: Precious Millicent Zvidi

Registration Number: H250137J

Process finished with exit code 0

RecordDemo.java

```
package lab09;

public class RecordDemo {

    // Record representing student information
    record Student(
            String name,
            String registrationNumber,
            String department
    ) {}
```

```
    public static void main(String[] args) {

        // Create student records
        Student student1 = new Student(
                "Precious Millicent Zvidi",
                "H250137J",
                "Software Engineering"
        );

        Student student2 = new Student(
                "Tendai",
                "H250138K",
                "Computer Science"
        );

        // Access record components
        System.out.println("Student 1 Details:");
        System.out.println("Name: " + student1.name());
        System.out.println("Registration Number: "
                + student1.registrationNumber());
        System.out.println("Department: "
                + student1.department());

        // Automatically generated toString()
        System.out.println("\nStudent 2 Details:");
        System.out.println(student2);

        // Compare records
        Student student3 = new Student(
                "Precious Millicent Zvidi",
                "H250137J",
                "Software Engineering"
        );

        System.out.println("\nAre student1 and student3 equal?");
        System.out.println(student1.equals(student3));
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56284" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab09.StaticNestedDemo

University: Harare Institute of Technology

Department: Software Engineering

Course: Object-Oriented Programming

Process finished with exit code 0

StaticNestedDemo.java

```
package lab09;

public class StaticNestedDemo {

    // Outer class
    static class University {

        private static String universityName =
                "Harare Institute of Technology";

        // Static nested class
        static class Department {

            private String departmentName;
            private String courseName;

            public Department(String departmentName,
                              String courseName) {
                this.departmentName = departmentName;
                this.courseName = courseName;
            }

            public void displayDetails() {
                System.out.println("University: " + universityName);
                System.out.println("Department: " + departmentName);
                System.out.println("Course: " + courseName);
            }
        }
    }

    public static void main(String[] args) {

        // Create a static nested class object directly
        University.Department department =
                new University.Department(
                        "Software Engineering",
                        "Object-Oriented Programming"
                );

        // Display department information
        department.displayDetails();
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56284" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -

Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab09.StaticNestedDemo

University: Harare Institute of Technology

Department: Software Engineering

Course: Object-Oriented Programming

Process finished with exit code 0

12.5 Conclusion

Lab 9 demonstrated useful Java language features that improve clarity, reduce repetitive code and support correct object comparison.

13. LAB 10: FILE I/O, LAMBDAS AND STREAMS

13.1 Objectives

The objectives of Lab 10 were to:

- Read from and write to files.
- Use Java file-handling classes.
- Understand functional interfaces.
- Implement lambda expressions.
- Process data using the Stream API.

13.2 Introduction

File input and output (I/O) allows a program to read data from files and save information for later use. Lambda expressions provide a concise way of implementing functional interfaces. The Stream API supports operations such as filtering, sorting, transforming and summarising data.

13.3 Practical Work Performed

File-handling exercises demonstrated how text could be written to and read from files. Exception handling was important because file operations may fail due to missing files, access restrictions or other I/O problems.

Lambda expressions were used to provide implementations of functional interfaces without writing lengthy anonymous classes. Stream operations were used to process collections of data through a sequence of operations.

13.4 Results and Observations

The exercises demonstrated that files can provide persistent storage beyond the lifetime of a running program. Lambda expressions reduce repetitive code when a short behaviour is required.

Streams make data processing more expressive by allowing operations to be linked into a pipeline. Appropriate exception handling is necessary to manage file-related errors.

FileReadingDemo.java

```
package lab10;

import java.io.FileReader;
import java.io.BufferedReader;
import java.io.IOException;

public class FileReadingDemo {

    public static void main(String[] args) {

        String fileName = "student_details.txt";

        System.out.println("Reading Student Details:");
        System.out.println();

        // Try-with-resources closes the reader automatically
        try (BufferedReader reader =
                     new BufferedReader(new FileReader(fileName))) {

            String line;

            // Read the file line by line
            while ((line = reader.readLine()) != null) {

                System.out.println(line);
            }

            System.out.println();
            System.out.println("File read successfully!");

        } catch (IOException e) {

            System.out.println("Error reading file: "
                    + e.getMessage());
```

```
            }
        }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56272" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab10.FileReadingDemo

Reading Student Details:

STUDENT DETAILS

Name: Precious Millicent Zvidi

Registration Number: H250137J

Department: Software Engineering

Institution: Harare Institute of Technology

File read successfully!

Process finished with exit code 0

FileStreamDemo.java

```
package lab10;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
```

```java
public class FileStreamDemo {

    public static void main(String[] args) {

        Path filePath = Path.of("student_marks.txt");

        // Student marks to be saved
        List<String> marks = List.of(
                "45", "78", "90", "62", "35", "88", "55"
        );

        try {

            // Write marks to a text file
            Files.write(filePath, marks);

            System.out.println("Student marks saved successfully.");

            // Read marks from the text file
            List<Integer> studentMarks = Files.readAllLines(filePath)
                    .stream()
                    .map(Integer::parseInt)
                    .toList();

            System.out.println("\nAll Student Marks:");
            System.out.println(studentMarks);

            // Filter passing marks
            List<Integer> passingMarks = studentMarks.stream()
                    .filter(mark -> mark >= 50)
                    .toList();

            System.out.println("\nPassing Marks:");
            System.out.println(passingMarks);

            // Count passing students
            long passCount = studentMarks.stream()
                    .filter(mark -> mark >= 50)
                    .count();

            System.out.println("\nNumber of Students Who Passed: "
                    + passCount);

            // Calculate average mark
            double average = studentMarks.stream()
                    .mapToInt(Integer::intValue)
                    .average()
                    .orElse(0);

            System.out.printf("Average Mark: %.2f%n", average);

            // Find highest mark
            int highest = studentMarks.stream()
                    .mapToInt(Integer::intValue)
                    .max()
                    .orElse(0);
```

```
            System.out.println("Highest Mark: " + highest);

            System.out.println("\nFile processing completed successfully.");

        } catch (IOException e) {

            System.out.println("File Error: " + e.getMessage());

        } catch (NumberFormatException e) {

            System.out.println("Invalid mark found in file.");
        }
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56274" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab10.FileStreamDemo

Student marks saved successfully.

All Student Marks:

[45, 78, 90, 62, 35, 88, 55]

Passing Marks:

[78, 90, 62, 88, 55]

Number of Students Who Passed: 5

Average Mark: 64.71

Highest Mark: 90

File processing completed successfully.

Process finished with exit code 0

FileWritingDemo.java

```java
package lab10;

import java.io.FileWriter;
import java.io.IOException;

public class FileWritingDemo {

    public static void main(String[] args) {

        String fileName = "student_details.txt";

        // Try-with-resources automatically closes the file
        try (FileWriter writer = new FileWriter(fileName)) {

            // Write student information to the file
            writer.write("STUDENT DETAILS\n");
            writer.write("--------------------------\n");
            writer.write("Name: Precious Millicent Zvidi\n");
            writer.write("Registration Number: H250137J\n");
            writer.write("Department: Software Engineering\n");
            writer.write("Institution: Harare Institute of Technology\n");

            System.out.println("File created successfully!");
            System.out.println("Student details written to "
                    + fileName);

        } catch (IOException e) {

            System.out.println("Error writing to file: "
                    + e.getMessage());
        }
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56277" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab10.FileWritingDemo

File created successfully!

Student details written to student_details.txt

Process finished with exit code 0

LambdaExpressionDemo.java

```java
package lab10;

public class LambdaExpressionDemo {

    // Functional interface
    @FunctionalInterface
    interface Calculator {
        int calculate(int a, int b);
    }

    // Another functional interface
    @FunctionalInterface
    interface Greeting {
        void greet(String name);
    }

    public static void main(String[] args) {

        // Lambda expression for addition
        Calculator addition = (a, b) -> a + b;

        // Lambda expression for multiplication
        Calculator multiplication = (a, b) -> a * b;

        // Lambda expression for subtraction
        Calculator subtraction = (a, b) -> a - b;

        // Lambda expression for greeting
        Greeting greeting = name ->
                System.out.println("Welcome, " + name + "!");

        System.out.println("Lambda Expression Results:");

        System.out.println("Addition: "
                + addition.calculate(10, 5));

        System.out.println("Multiplication: "
                + multiplication.calculate(10, 5));

        System.out.println("Subtraction: "
                + subtraction.calculate(10, 5));

        System.out.println();

        greeting.greet("Precious");

        System.out.println("\nProgram completed successfully.");
    }
}
```

```
"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56279" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -
```

Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab10.LambdaExpressionDemo

Lambda Expression Results:

Addition: 15

Multiplication: 50

Subtraction: 5

Welcome, Precious!

Program completed successfully.

Process finished with exit code 0

StreamDemo.java

```
package lab10;

import java.util.Arrays;
import java.util.List;

public class StreamDemo {

    public static void main(String[] args) {

        // List of student marks
        List<Integer> marks = Arrays.asList(
                45, 78, 90, 62, 35, 88, 55
        );

        System.out.println("Original Student Marks:");
        System.out.println(marks);

        // Filter marks greater than or equal to 50
        System.out.println("\nPassing Marks:");

        marks.stream()
                .filter(mark -> mark >= 50)
                .forEach(System.out::println);
```

```
        // Sort marks in ascending order
        System.out.println("\nSorted Marks:");

        marks.stream()
                .sorted()
                .forEach(System.out::println);

        // Count students who passed
        long passedStudents = marks.stream()
                .filter(mark -> mark >= 50)
                .count();

        System.out.println("\nNumber of Students Who Passed: "
                + passedStudents);

        // Calculate average mark
        double average = marks.stream()
                .mapToInt(Integer::intValue)
                .average()
                .orElse(0);

        System.out.printf("Average Mark: %.2f%n", average);

        // Find highest mark
        int highest = marks.stream()
                .mapToInt(Integer::intValue)
                .max()
                .orElse(0);

        System.out.println("Highest Mark: " + highest);

        System.out.println("\nProgram completed successfully.");
    }
}
```

"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.3\lib\idea_rt.jar=56281" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath C:\Users\user\IdeaProjects\HIT-Java-OOP-Labs\out\production\HIT-Java-OOP-Labs lab10.StreamDemo

Original Student Marks:

[45, 78, 90, 62, 35, 88, 55]

Passing Marks:

78

90

62

88

55

Sorted Marks:

35

45

55

62

78

88

90

Number of Students Who Passed: 5

Average Mark: 64.71

Highest Mark: 90

Program completed successfully.

Process finished with exit code 0

13.5 Conclusion

Lab 10 demonstrated how file handling supports data persistence and how lambdas and streams make Java data-processing operations more concise.

14. LAB 11: DESIGN PATTERNS AND LIBRARY MANAGEMENT SYSTEM MINI-PROJECT

14.1 Objectives

The objectives of Lab 11 were to:

- Understand common software design patterns.
- Apply the Singleton, Factory, Strategy and Observer patterns where appropriate.
- Integrate Java OOP concepts in one application.
- Design a small Library Management System.
- Test the system's functionality and handle invalid operations.

14.2 Introduction

Design patterns are reusable approaches to common software design problems. They provide established ways of structuring classes and their interactions.

The Library Management System mini-project applies concepts from the earlier laboratories to a practical problem. A library needs to organise its items, register members, track borrowing activities and manage the availability of items.

14.3 Design Patterns

14.3.1 Singleton Pattern

The Singleton pattern is used when a class should provide one shared instance. It can be useful for managing a central service or configuration object.

14.3.2 Factory Pattern

The Factory pattern separates object creation from the code that uses the objects. It can be useful when creating different types of library items without requiring the calling code to construct each concrete type directly.

14.3.3 Strategy Pattern

The Strategy pattern allows different algorithms or rules to be selected without changing the code that uses them. For example, a system could use different borrowing policies for different types of members.

14.3.4 Observer Pattern

The Observer pattern allows registered listeners to receive notifications when a particular event occurs. In a library application, it could be used to notify interested components when an item or loan status changes.

The presence of a pattern in this report describes its purpose; only patterns actually implemented in the submitted source code should be claimed as completed practical work.

14.4 Description of the Mini-Project

The mini-project is a Library Management System developed in Java. Its purpose is to demonstrate how OOP concepts can be combined to organise library information and borrowing operations.

The application can be designed around library items, members and borrowing records. Depending on the implemented features, users may be able to add items, register members, search for items, borrow available items and return borrowed items.

The system provides an opportunity to apply concepts such as encapsulation, inheritance, abstraction, polymorphism, interfaces, collections and exception handling in a single program.

14.5 Main Components

The following components represent a suitable structure for a Library Management System. The exact names and responsibilities should be adjusted to match the classes in the submitted project.

Library item class: Stores shared information about an item, such as its identifier, title and availability.

Book class: Represents a book and can contain book-specific information, such as its author.

DVD class: Represents a DVD or another supported library item type, with its own relevant details.

Member class: Stores information about a registered library member.

Loan class: Represents a borrowing transaction and may store the member, borrowed item, dates and return status.

Borrowing interface: Defines the operations or contract required for borrowing, if such an interface is implemented.

Exception-handling components: Handle invalid operations, unavailable items and other error conditions.

Collections: Store and retrieve items, members and loan records while the program is running.

File-handling components: Save or load records if file persistence is included in the implementation.

Menu or main class: Provides the entry point and allows users to select the available operations.

14.6 Implementation and Testing

The project was opened and run in IntelliJ IDEA. The code was organised in the project source directory and the application was checked through its displayed menu and available operations.

During testing, the message **“No library item available”** was observed. This indicates that the application reached a check for item availability, but it does not by itself prove that the borrowing process works correctly. The application should also be tested after an item has been added and its availability confirmed.

The following tests should be performed where the relevant features are implemented:

Test case	Expected result
Start the application	The menu or start screen is displayed.
Add a library item	The item is stored and can be found.
Register a member	The member's information is stored.
Search for an item	Matching item details are displayed.
Borrow an available item	The borrowing operation succeeds and availability is updated.
Attempt to borrow an unavailable item	The operation is rejected with a clear message.
Return a borrowed item	The return is recorded and availability is updated.
Enter invalid information	The application handles the input appropriately.

The test results should be recorded after running each test. If a feature is not implemented or has not been tested, it should be identified as such rather than reported as successful.

14.7 Results and Discussion

The mini-project brings together concepts from the earlier practicals. Classes and objects represent library entities, encapsulation protects internal data, inheritance can support different item types, and polymorphism allows different types to be handled through a common parent type.

Collections can be used to manage records, while exception handling helps the application respond to invalid operations. File handling can provide persistent storage if it is included in the submitted implementation.

The observed availability message also highlights the importance of testing a system with both valid and invalid scenarios. A program should be tested with suitable data to confirm that its operations behave as intended.

14.8 Conclusion

Lab 11 provides an opportunity to integrate Java programming concepts in a practical application. The Library Management System demonstrates how object-oriented design can be used to represent library items, members and borrowing activities.

The mini-project also highlights the importance of testing, clear error messages, organised source code and appropriate use of design patterns. Further testing and verification of each implemented feature are necessary before the project is considered complete.

15. OVERALL RESULTS AND LEARNING OUTCOMES

The laboratory series introduced progressively more advanced Java programming concepts. The earlier practicals established an understanding of classes, objects, constructors and data protection. The middle practicals focused on inheritance, polymorphism, abstraction, exception handling and collections.

The later practicals introduced nested types, enums, records, file operations, lambda expressions and streams. These concepts provided additional techniques for organising and processing data.

The Lab 11 mini-project brought together several of these concepts in a single application. It demonstrated the importance of planning class responsibilities, organising code, checking program behaviour and handling errors.

The laboratory series also reinforced the importance of compiling and testing programs regularly, keeping source files organised and documenting results clearly.

16. GENERAL CONCLUSION

The Java Object-Oriented Programming laboratory series provided practical exposure to the principles and features of Java programming. Each laboratory introduced concepts that contributed to the development of structured and reusable software.

The exercises progressed from basic object creation to more advanced programming techniques, including inheritance, polymorphism, abstraction, collections, exception handling and file processing. The Library Management System mini-project provided an opportunity to combine these concepts in a practical application.

Overall, the practicals established a foundation for further study and software development. Continued practice, testing and refinement of the submitted code will help strengthen the ability to design reliable and maintainable Java applications.